
Computed-Torque Controller

Cable-Driven 2R Planar Manipulator — PyDrake Implementation

Isaac Sim Robotics Project

April 2026

Key Results at a Glance

- **Tracking RMS error:** < 1 mm on $20\text{ mm} \times 160\text{ mm}$ rectangle trajectory
- **Controller:** Inverse-dynamics (computed torque) with joint-space PD + feedforward
- **Closed-loop bandwidth:** $\omega_n = \sqrt{K_p} \approx 20\text{ rad/s}$, $\zeta = 1.0$ (critically damped)
- **Simulation framework:** PyDrake MultibodyPlant — analytical dynamics, Meshcat 3D visualization
- **Cable actuation:** Joint-2 driven via HTD 5M belt with pulley radius $r_p = 47.7\text{ mm}$
- **Architecture:** Two-plant Drake pattern (physics + controller) with diagram-based signal routing

Contents

1	Algorithm Overview	4
1.1	What the Simulation Actually Does	4
1.2	Algorithm Flow — Step by Step	4
1.3	What the Cable Is (and Isn't)	4
2	Source File Architecture	6
2.1	File Dependency Map	6
2.2	Module Responsibilities	6
2.3	Design Principle: Engine-Agnostic Core	7
2.4	Import Graph — Main Script	7
3	System Description	8
3.1	Physical Robot	8
3.2	Schematic Diagram	8
3.3	CupManipulatorTendon — Configuration Class	8
3.4	CupManipulatorIK — Inverse Kinematics Helper	9

4	Dynamics — Manipulator Equation of Motion	10
4.1	Standard Manipulator Dynamics	10
4.2	2R Planar Arm — Analytical Mass Matrix	10
5	Forward Kinematics and Jacobian	11
5.1	Forward Kinematics	11
5.2	Analytical Jacobian	11
5.3	Inverse Kinematics — 2R Closed-Form Solution	11
6	Computed-Torque Control Law	12
6.1	Theory: Feedback Linearization	12
6.2	Desired Acceleration with Feedforward	12
6.3	Closed-Loop Error Dynamics	13
6.4	Full Controller Code	13
7	Pure NumPy Controller Engine	14
7.1	CTControllerOutput Dataclass	14
7.2	ComputedTorqueController Class	14
7.3	EE-to-Joint Reference Conversion	15
8	PyDrake Architecture — Two-Plant Diagram Pattern	16
8.1	Drake Diagram Overview	16
8.2	ComputedTorqueController — Drake LeafSystem	17
9	Trajectory Generation	19
9.1	C ² Cubic Spline Trajectories	19
9.2	Rectangle Trajectory with Velocity Profiling	19
9.3	Circle and Line Trajectories	20
9.4	Move-to-Start Preamble	20
10	Cable Routing and Tension Decomposition	21
10.1	Belt-Pulley Kinematics	21
10.2	Cable Visualization via DrakeCablePlant	21
10.3	Pulley Class Hierarchy	21
10.4	Tangent-Line Geometry	22
10.5	CableRig — Route Assembly	22
10.6	CableSpring — Optional Compliance	23
11	Simulation — Builder Pattern	24
11.1	ComputedTorqueSimulation Class Hierarchy	24
11.2	Builder Pattern — Step-by-Step	24
11.3	IKPDSimulation — Base Closed-Loop Class	24
11.4	ComputedTorqueSimulation — CT Subclass	25
11.5	Initialization Priority	25
11.6	Cable Visualization Loop	25
11.7	Zero-Mass Body Patching	26
12	Simulation Modes and CLI	26
13	Visualization — Meshcat	27
13.1	Coordinate Frame Visualization	27
13.2	Camera Control	28

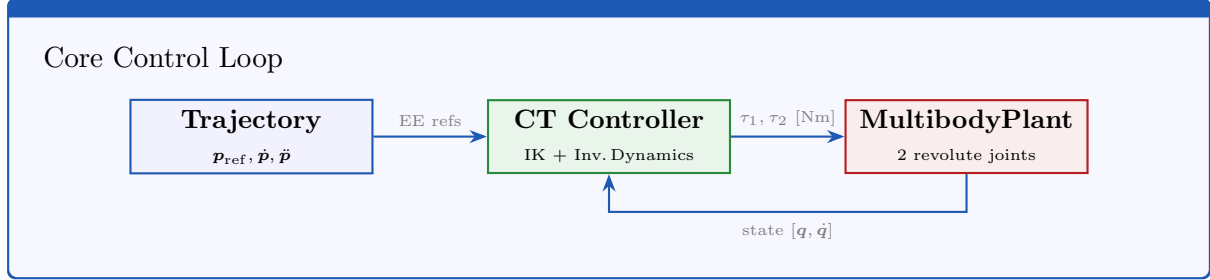
14 Results and Plotting	28
14.1 3×3 Diagnostic Plot	28
14.2 Key Metrics	29
15 Notation Summary	29
16 Running the Simulation	29
17 Summary	30

1 Algorithm Overview

This section gives the top-level picture before the detailed derivations that follow.

1.1 What the Simulation Actually Does

The simulation runs a **single Drake MultibodyPlant** loaded from a URDF with **two revolute joints**. A computed-torque controller (a Drake **LeafSystem**) reads the plant state, computes joint torques via inverse dynamics, and writes them back to the plant's actuation port. The closed-loop runs at 500 Hz (2 ms timestep).



1.2 Algorithm Flow — Step by Step

Every simulation timestep the controller executes:

1. **Read state** $(q, \dot{q})$ from the MultibodyPlant
2. **Evaluate trajectory** $\rightarrow$ desired EE position p_{ref} , velocity $\dot{p}_{\text{ref}}$, acceleration $\ddot{p}_{\text{ref}}$
3. **Inverse kinematics:** $p_{\text{ref}} \xrightarrow{\text{2R analytical}} q_{\text{des}}$
4. **Differential IK:** $\dot{p}_{\text{ref}} \xrightarrow{J^{-1}} \dot{q}_{\text{ref}}, \quad \ddot{p}_{\text{ref}} \xrightarrow{J^{-1}} \ddot{q}_{\text{ref}}$
5. **PD + feedforward:** $a_{\text{des}} = \ddot{q}_{\text{ref}} + K_p(q_{\text{des}} - q) + K_d(\dot{q}_{\text{ref}} - \dot{q})$
6. **Inverse dynamics:** $\tau = M(q) a_{\text{des}} + h(q, \dot{q})$ via Drake's `CalcInverseDynamics`
7. **Saturate and apply:** $\tau_{\text{clip}} = \text{clip}(\tau, \pm\tau_{\text{max}}) \rightarrow$ plant actuation port

1.3 What the Cable Is (and Isn't)

The cable/belt is NOT modeled as a physical element. The Drake MultibodyPlant contains only two rigid revolute joints—there is no spring, cable, or pulley in the physics simulation. The belt is treated as a **rigid tendon**: an inextensible, massless transmission between motor and joint.

The controller outputs **joint torques** $[\tau_1, \tau_2]$ in Nm, which are applied directly to the plant's actuation port. There is no cable compliance, no pulley friction, and no belt elasticity in the dynamics.

Cable tensions appear in the code as a **post-hoc decomposition** for logging and feasibility checking:

$$F_{\text{net}} = \frac{\tau_2}{r_p}, \quad T_{\text{green}} = \max(F_{\text{net}}, 0), \quad T_{\text{red}} = \max(-F_{\text{net}}, 0)$$

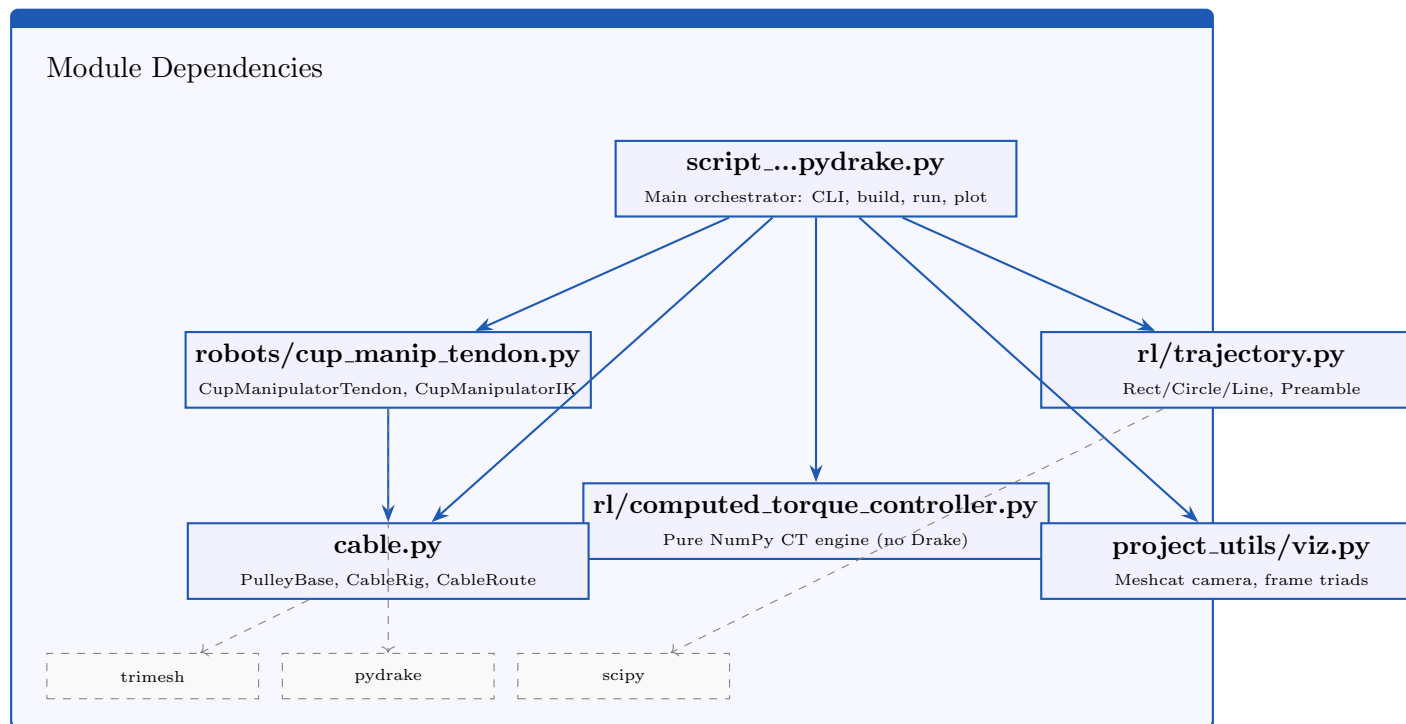
This decomposition enforces the physical constraint that cables can only pull—if $T_{\text{green}} > 0$ and $T_{\text{red}} > 0$ simultaneously, the tendon assumption is violated. But the plant simulation itself never sees cable tensions; it only receives τ_1, τ_2 .

The cable **visualization** (coloured polylines in Meshcat) is likewise separate from the physics. A headless `DrakeCablePlant` computes pulley tangent geometry via FK for rendering only.

2 Source File Architecture

This implementation spans six Python modules. Each module has a distinct responsibility, and they compose through Drake's `DiagramBuilder` wiring pattern.

2.1 File Dependency Map



2.2 Module Responsibilities

Table 1: Source file roles and key exports.

File	Key Exports / Role
<code>script_...pydrake.py</code> (~1600 lines)	<code>IKPDSimulation</code> , <code>ComputedTorqueSimulation</code> , CLI parser, <code>_SIM_REGISTRY</code> dispatch
<code>robots/cup_manipulator_tendon.py</code> (~500)	<code>CupManipulatorTendon</code> (URDF loading, FK, weld, actuators), <code>CupManipulatorIK</code> (2R analytical IK, Jacobians)
<code>rl/computed_torque_controller.py</code> (~170)	<code>ComputedTorqueController.compute()</code> , <code>ik_to_joint_space_references()</code> — pure NumPy, engine-agnostic
<code>rl/trajectory.py</code> (~300)	<code>RectTrajectory</code> , <code>CircleTrajectory</code> , <code>LineTrajectory</code> , <code>MoveToStartSpline</code> , <code>PreambleTrajectorySource</code>
<code>cable.py</code> (~800)	<code>PulleyBase</code> hierarchy, <code>CableRig</code> , <code>CableRoute</code> , <code>CableSpring</code> , tangent geometry
<code>project_utils/viz.py</code> (~200)	<code>set_meshcat_camera_spherical()</code> , <code>add_frames_to_meshcat()</code> , frame triads

2.3 Design Principle: Engine-Agnostic Core

A key architectural decision is the separation between **engine-agnostic** and **engine-specific** modules:

- `rl/computed_torque_controller.py` uses **only NumPy**—no Drake imports. It accepts M , h , and state vectors as plain arrays. This allows the *same controller* to run in both PyDrake and Isaac Sim.
- `rl/trajectory.py` uses **only SciPy**—no Drake imports. Trajectories are portable across simulation engines.
- `robots/cup_manipulator_tendon.py` and the main script are Drake-specific, using `MultibodyPlant`, `LeafSystem`, and `DiagramBuilder`.
- `cable.py` uses Drake `MultibodyPlant` only for forward kinematics (a headless plant); the tangent-line geometry is pure NumPy.

2.4 Import Graph — Main Script

The main script imports are organized by layer:

```

1  # --- Engine layer (Drake) ---
2  from pydrake.all import (
3      DiagramBuilder, Simulator, StartMeshcat,
4      MultibodyPlant, Parser, AddMultibodyPlantSceneGraph,
5      MeshcatVisualizer, MeshcatVisualizerParams, Role,
6      LogVectorOutput,                # data logging
7      ConstantVectorSource,          # trajectory sources
8      RigidTransform, RotationMatrix,
9  )
10
11 # --- Robot layer ---
12 from robots.cup_manipulator_tendon import CupManipulatorTendon
13
14 # --- Engine-agnostic controller ---
15 from rl.computed_torque_controller import (
16     ComputedTorqueController, ik_to_joint_space_references)
17
18 # --- Engine-agnostic trajectory ---
19 from rl.trajectory import (
20     RectTrajectory, CircleTrajectory, LineTrajectory,
21     PreambleTrajectorySource, build_move_to_start)
22
23 # --- Cable geometry ---
24 from cable import DrakeCablePlant, CableRig
25
26 # --- Visualization ---
27 from project_utils.viz import (
28     set_meshcat_camera_spherical, add_frames_to_meshcat)

```

Listing 1: Import structure of the main simulation script.

3 System Description

3.1 Physical Robot

The cable-driven cup manipulator is a **2-DOF planar revolute-revolute (2R)** arm:

- **Joint 1** (`link1_base`): Direct-drive motor at the base — torque τ_1 applied directly.
- **Joint 2** (`link2_link1`): Cable-driven via an HTD5M timing belt routed through idler pulleys. The motor torque τ_{motor} maps to joint torque via the pulley radius:

$$\tau_2 = F_{\text{net}} \cdot r_p, \quad F_{\text{net}} = T_{\text{green}} - T_{\text{red}}$$

Table 2: Robot physical parameters (from URDF).

Parameter	Symbol	Value	Unit
Link 1 length	L_1	335	mm
Link 2 length	L_2	190	mm
Link 1 mass	m_1	5.313	kg
Link 2 mass	m_2	0.520	kg
Link 1 inertia (I_{zz})	I_{1z}	0.061	$\text{kg} \cdot \text{m}^2$
Base mass	m_0	0.896	kg
Pulley radius (HTD 5M 60T)	r_p	47.746	mm
EE offset on link 2	—	[190, 0, 51.5]	mm

3.2 Schematic Diagram

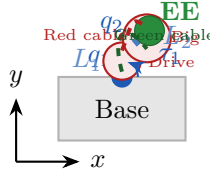


Figure 1: 2R cable-driven manipulator. Joint 1 is direct-drive; Joint 2 uses a cable routed through drive, idler, and big pulleys.

3.3 CupManipulatorTendon — Configuration Class

The robot is represented by a frozen dataclass that stores all physical parameters, URDF paths, and joint configurations. The class `CupManipulatorTendon` (in `robots/cup_manipulator_tendon.py`) wraps this config and provides Drake-facing methods:

```

1  class CupManipulatorTendon(RobotBase):
2      # --- Joint names (matches URDF parse order) ---
3      JT1_NAME   = "link1_base"           # q1: direct-drive
4      JT2_NAME   = "link2_link1"         # q2: cable-driven
5      ACT1_NAME  = "tau_link1_base"
6      ACT2_NAME  = "tau_link2_link1"
7
8      # --- End-effector frame offset on link2 ---
9      EE_XYZ_LINK2 = [0.19, 0.0, 0.0515] # [m]
10     EE_FRAME_NAME = "tendon_ee"

```



```

11  # --- Pulley parameters (HTD 5M 60T belt) ---
12  PULLEY_RADIUS = 60 * 0.005 / (2 * np.pi) # ~0.04775 m
13
14  # Key methods:
15  def load_urdf_to_plant(self, plant, parser): ...
16  def weld_base_to_world(self, plant, pos, orient): ...
17  def add_end_effector_frame(self, plant): ...
18  def add_joint_actuators(self, plant): ...
19  def get_state_from_plant(self, plant, ctx): # [q1, q2, dq1, dq2]
20  def compute_ik_analytical(self, plant, target_xy, q_seed): ...
21  def init_cable_rig(self, urdf_path, assets_dir): ...
22

```

Listing 2: Key constants and methods of CupManipulatorTendon.

Drake joint ordering caveat. Drake's `GetJointIndices()` returns joints in URDF parse order. For the cable manipulator URDF this is `[link2.link1, link1_base] = [q2, q1]`, *not* `[q1, q2]`. The `get_state_from_plant()` and `set_state_in_plant()` helpers re-map to user order `[q1, q2]` automatically.

3.4 CupManipulatorIK — Inverse Kinematics Helper

The `CupManipulatorIK` class (inner member `self.ik` of the manipulator) provides all kinematic queries:

```

1  class CupManipulatorIK:
2      def get_link_lengths(self, plant) -> Tuple[float, float]:
3          """Extract (L1, L2) from FK at q=0."""
4          # L1 = |J1_origin - J2_origin| in XY plane
5          # L2 = |J2_origin - EE_frame| in XY plane
6
7      def analytical(self, plant, target_xy, q_seed):
8          """Closed-form 2R IK: returns (q_best, success)."""
9          return self._solve_2r_core(L1, L2, target_xy, q_seed)
10
11     def jacobian(self, plant, context) -> np.ndarray:
12         """Drake automatic Jacobian via
13             CalcJacobianTranslationalVelocity."""
14         # Returns J in R^{2x2}: dp_EE = J(q) * dq
15
16     def jacobian_analytical(self, plant, context) -> np.ndarray:
17         """Closed-form planar Jacobian (no Drake call)."""
18         # J = [[-L1*s1 - L2*s12, -L2*s12],
19         #       [ L1*c1 + L2*c12,  L2*c12]]
20
21     def velocity(self, plant, context, ee_vel, damping=1e-4):
22         """Pseudo-inverse velocity IK: dq = pinv(J) @ ee_vel."""

```

Listing 3: IK helper methods.

► Equations implemented by the above code

Link lengths are extracted once at construction time via forward kinematics at $\mathbf{q} = \mathbf{0}$:

$$L_1 = \|\mathbf{p}_{J2} - \mathbf{p}_{J1}\|_{xy}, \quad L_2 = \|\mathbf{p}_{EE} - \mathbf{p}_{J2}\|_{xy}$$

The `jacobian()` method uses Drake's articulated-body Jacobian (exact for the URDF model), while `jacobian_analytical()` computes the planar 2×2 matrix directly. Both produce identical results for planar motion.

4 Dynamics — Manipulator Equation of Motion

4.1 Standard Manipulator Dynamics

The equations of motion for a rigid-body manipulator are:

$$\boxed{\mathbf{M}(\mathbf{q}) \ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau}} \quad (1)$$

where:

- $\mathbf{q} = [q_1, q_2]^\top$ — joint positions [rad]
- $\dot{\mathbf{q}} = [\dot{q}_1, \dot{q}_2]^\top$ — joint velocities [rad/s]
- $\mathbf{M}(\mathbf{q}) \in \mathbb{R}^{2 \times 2}$ — symmetric positive-definite **mass (inertia) matrix**
- $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \dot{\mathbf{q}} \in \mathbb{R}^2$ — **Coriolis and centrifugal** terms
- $\mathbf{g}(\mathbf{q}) \in \mathbb{R}^2$ — **gravitational** torques
- $\boldsymbol{\tau} = [\tau_1, \tau_2]^\top$ — applied joint torques [Nm]

We define the **bias vector** for compactness:

$$\mathbf{h}(\mathbf{q}, \dot{\mathbf{q}}) \triangleq \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) \quad (2)$$

so that (1) becomes $\mathbf{M}(\mathbf{q}) \ddot{\mathbf{q}} + \mathbf{h}(\mathbf{q}, \dot{\mathbf{q}}) = \boldsymbol{\tau}$.

4.2 2R Planar Arm — Analytical Mass Matrix

For a planar 2R arm with link lengths L_1, L_2 , link masses m_1, m_2 , and link inertias I_1, I_2 :

$$\mathbf{M}(\mathbf{q}) = \begin{bmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \end{bmatrix} \quad (3)$$

where:

$$m_{11} = I_1 + I_2 + m_1 r_1^2 + m_2 (L_1^2 + r_2^2 + 2L_1 r_2 \cos q_2) \quad (4)$$

$$m_{12} = m_{21} = I_2 + m_2 (r_2^2 + L_1 r_2 \cos q_2) \quad (5)$$

$$m_{22} = I_2 + m_2 r_2^2 \quad (6)$$

where r_1, r_2 are the distances from each joint to the respective link's center of mass.

In **PyDrake**, the mass matrix $\mathbf{M}(\mathbf{q})$ is computed automatically by the `MultibodyPlant` via `plant.CalcMassMatrix(context)`. This uses the full URDF inertia tensors, including off-diagonal terms and 3D contributions projected onto the 2D plane — which is more

accurate than the analytical formula above.

5 Forward Kinematics and Jacobian

5.1 Forward Kinematics

The end-effector position in the world frame for the horizontal SCARA-like configuration:

$$\mathbf{p}_{EE} = \begin{bmatrix} L_1 \cos q_1 + L_2 \cos(q_1 + q_2) \\ L_1 \sin q_1 + L_2 \sin(q_1 + q_2) \end{bmatrix} \quad (7)$$

5.2 Analytical Jacobian

The Jacobian mapping joint velocities to EE velocities is:

$$\mathbf{J}(\mathbf{q}) = \frac{\partial \mathbf{p}_{EE}}{\partial \mathbf{q}} = \begin{bmatrix} -L_1 \sin q_1 - L_2 \sin(q_1 + q_2) & -L_2 \sin(q_1 + q_2) \\ L_1 \cos q_1 + L_2 \cos(q_1 + q_2) & L_2 \cos(q_1 + q_2) \end{bmatrix} \quad (8)$$

so that $\dot{\mathbf{p}}_{EE} = \mathbf{J}(\mathbf{q}) \dot{\mathbf{q}}$.

Denoting $s_1 = \sin q_1$, $c_1 = \cos q_1$, $s_{12} = \sin(q_1 + q_2)$, $c_{12} = \cos(q_1 + q_2)$:

$$\mathbf{J} = \begin{bmatrix} -L_1 s_1 - L_2 s_{12} & -L_2 s_{12} \\ L_1 c_1 + L_2 c_{12} & L_2 c_{12} \end{bmatrix} \quad (9)$$

```

1 # Analytical Jacobian at q_des
2 q1d, q2d = q_des
3 s1 = np.sin(q1d); c1 = np.cos(q1d)
4 s12 = np.sin(q1d + q2d); c12 = np.cos(q1d + q2d)
5 J = np.array([
6     [-L1 * s1 - L2 * s12, -L2 * s12],
7     [L1 * c1 + L2 * c12, L2 * c12],
8 ])

```

Listing 4: Analytical Jacobian — `rl/computed_torque_controller.py`

► Equations implemented by the above code

The code builds the 2×2 Jacobian matrix from (8) using compact trigonometric notation. The pseudo-inverse $\mathbf{J}^+ = \mathbf{J}^{-1}$ (since $\mathbf{J}$ is square and non-singular away from singularities) is then used to map Cartesian references to joint space:

$$\dot{\mathbf{q}}_{\text{ref}} = \mathbf{J}^{-1} \dot{\mathbf{x}}_{\text{ref}}, \quad \ddot{\mathbf{q}}_{\text{ref}} \approx \mathbf{J}^{-1} \ddot{\mathbf{x}}_{\text{ref}}$$

where the $\dot{\mathbf{J}} \dot{\mathbf{q}}$ bias term is dropped (order $O(\dot{q}^2)$, small for slow trajectories).

5.3 Inverse Kinematics — 2R Closed-Form Solution

For a target $\mathbf{p}^* = [x^*, y^*]^\top$, the analytical IK solution is:

$$\cos q_2 = \frac{x^{*2} + y^{*2} - L_1^2 - L_2^2}{2 L_1 L_2} \quad (10)$$

$$q_2 = \pm \arccos(\cos q_2) \quad (\text{elbow up/down}) \quad (11)$$

$$q_1 = \text{atan2}(y^*, x^*) - \text{atan2}(L_2 \sin q_2, L_1 + L_2 \cos q_2) \quad (12)$$

Both elbow solutions are evaluated and the one *closest to the seed* $\mathbf{q}_{\text{seed}}$ is returned:

```

1 def _solve_2r_core(L1, L2, target_xy, q_seed, q2_limit):
2     x_t, y_t = target_xy
3     cos_q2 = (x_t**2 + y_t**2 - L1**2 - L2**2) / (2 * L1 * L2)
4     cos_q2 = np.clip(cos_q2, -1.0, 1.0)
5     candidates = []
6     for sign in (+1, -1):
7         q2 = sign * np.arccos(cos_q2)
8         q1 = np.arctan2(y_t, x_t) - np.arctan2(
9             L2 * np.sin(q2), L1 + L2 * np.cos(q2))
10        candidates.append(np.array([q1, q2]))
11    # Pick solution closest to seed
12    best = min(candidates, key=lambda c: np.sum((c - q_seed)**2))
13    return best, True

```

Listing 5: 2R IK solver — robots/cup_manipulator_tendon.py

► Equations implemented by the above code

Equations (10)–(12) are the standard closed-form inverse kinematics for a 2R planar arm. The seed-selection strategy ensures continuity along a trajectory:

$$\mathbf{q}^* = \arg \min_{\mathbf{q} \in \{\mathbf{q}^+, \mathbf{q}^-\}} \|\mathbf{q} - \mathbf{q}_{\text{seed}}\|^2$$

6 Computed-Torque Control Law

6.1 Theory: Feedback Linearization

The **computed-torque** (CT) controller achieves *feedback linearization* of the nonlinear manipulator dynamics. Given a desired joint-space acceleration $\mathbf{a}_{\text{des}}$, the control law cancels the nonlinear terms exactly:

$$\tau = M(\mathbf{q}) \mathbf{a}_{\text{des}} + \mathbf{h}(\mathbf{q}, \dot{\mathbf{q}}) \quad (13)$$

Substituting (13) into the dynamics (1):

$$M(\mathbf{q}) \ddot{\mathbf{q}} + \mathbf{h}(\mathbf{q}, \dot{\mathbf{q}}) = M(\mathbf{q}) \mathbf{a}_{\text{des}} + \mathbf{h}(\mathbf{q}, \dot{\mathbf{q}}) \quad (14)$$

$$M(\mathbf{q}) \ddot{\mathbf{q}} = M(\mathbf{q}) \mathbf{a}_{\text{des}} \quad (15)$$

$$\ddot{\mathbf{q}} = \mathbf{a}_{\text{des}} \quad (16)$$

This is exact cancellation, reducing the controlled system to a **double integrator**:

$$\ddot{\mathbf{q}} = \mathbf{a}_{\text{des}}$$

6.2 Desired Acceleration with Feedforward

The desired acceleration includes both **feedforward** (reference trajectory) and **feedback** (PD error correction):

$$\mathbf{a}_{\text{des}} = \ddot{\mathbf{q}}_{\text{ref}} + K_p (\mathbf{q}_{\text{des}} - \mathbf{q}) + K_d (\dot{\mathbf{q}}_{\text{ref}} - \dot{\mathbf{q}}) \quad (17)$$

where:

- $\mathbf{q}_{\text{des}}$ — desired joint position from IK
- $\dot{\mathbf{q}}_{\text{ref}}$ — desired joint velocity: $\dot{\mathbf{q}}_{\text{ref}} = \mathbf{J}^{-1} \dot{\mathbf{x}}_{\text{ref}}$
- $\ddot{\mathbf{q}}_{\text{ref}}$ — desired joint acceleration: $\ddot{\mathbf{q}}_{\text{ref}} \approx \mathbf{J}^{-1} \ddot{\mathbf{x}}_{\text{ref}}$
- K_p, K_d — scalar proportional and derivative gains

6.3 Closed-Loop Error Dynamics

Define the tracking error $\mathbf{e} = \mathbf{q} - \mathbf{q}_{\text{des}}$. Since $\ddot{\mathbf{q}} = \mathbf{a}_{\text{des}}$:

$$\ddot{\mathbf{e}} + K_d \dot{\mathbf{e}} + K_p \mathbf{e} = \mathbf{0} \quad (18)$$

This is a decoupled **second-order linear ODE** for each joint. The characteristic equation is:

$$s^2 + K_d s + K_p = 0 \quad (19)$$

with natural frequency and damping ratio:

$$\omega_n = \sqrt{K_p}, \quad \zeta = \frac{K_d}{2\omega_n} = \frac{K_d}{2\sqrt{K_p}} \quad (20)$$

For $K_p = 400$, $K_d = 40$: $\omega_n = 20$ rad/s, $\zeta = 1.0$ (**critically damped**). The error decays as $e(t) \sim (c_1 + c_2 t) e^{-\omega_n t}$ with time constant $T = 1/\omega_n = 50$ ms.

6.4 Full Controller Code

```

1  def compute(self, q, q_dot, q_des, q_dot_ref, q_ddot_ref, M, h):
2      """
3      a_des = q_ddot_ref + Kp*(q_des - q) + Kd*(q_dot_ref - q_dot)
4      tau   = M(q) @ a_des + h(q, q_dot)
5      """
6      a_des = q_ddot_ref + self.Kp * (q_des - q) \
7              + self.Kd * (q_dot_ref - q_dot)
8      tau_raw = M @ a_des + h
9
10     # Cable tension decomposition for joint 2
11     F_net = tau_raw[1] / self.r_p
12     T_green = max(F_net, 0.0) # retracting side
13     T_red   = max(-F_net, 0.0) # extending side
14     tau2_cmd = (T_green - T_red) * self.r_p
15
16     tau_clip = np.clip(
17         np.array([tau_raw[0], tau2_cmd]),
18         -self.tau_max, self.tau_max
19     )
20     return CTControllerOutput(tau_clip=tau_clip, tau_raw=tau_raw,
        ...)
```

Listing 6: Computed-torque law — rl/computed_torque_controller.py

► Equations implemented by the above code

The controller implements equations (13) and (17) in three steps:

Step 1 — PD + Feedforward acceleration:

$$\mathbf{a}_{\text{des}} = \ddot{\mathbf{q}}_{\text{ref}} + K_p (\mathbf{q}_{\text{des}} - \mathbf{q}) + K_d (\dot{\mathbf{q}}_{\text{ref}} - \dot{\mathbf{q}})$$

Step 2 — Inverse dynamics:

$$\boldsymbol{\tau}_{\text{raw}} = \mathbf{M}(\mathbf{q}) \mathbf{a}_{\text{des}} + \mathbf{h}(\mathbf{q}, \dot{\mathbf{q}})$$

Step 3 — Cable tension decomposition (joint 2):

$$F_{\text{net}} = \frac{\tau_2}{r_p}, \quad T_{\text{green}} = \max(F_{\text{net}}, 0), \quad T_{\text{red}} = \max(-F_{\text{net}}, 0)$$

$$\tau_{2,\text{cmd}} = (T_{\text{green}} - T_{\text{red}}) \cdot r_p$$

Finally, the torque is clipped: $\tau_{\text{clip}} = \text{clip}(\boldsymbol{\tau}, -\tau_{\text{max}}, +\tau_{\text{max}})$.

7 Pure NumPy Controller Engine

The engine-agnostic controller lives in `rl/computed_torque_controller.py`. It contains **zero Drake imports**—only NumPy—so it can be shared between PyDrake and Isaac Sim.

7.1 CTControllerOutput Dataclass

```

1 @dataclass
2 class CTControllerOutput:
3     tau_clip: np.ndarray      # [2] clipped torques [Nm]
4     tau_raw: np.ndarray      # [2] pre-saturation torques [Nm]
5     T_green: float           # retracting cable tension [N]
6     T_red: float             # extending cable tension [N]
7     q_des: np.ndarray        # [2] desired joint positions [rad]
8     a_des: np.ndarray        # [2] desired joint accelerations [rad/s
    ^2]
```

Listing 7: Output bundle from each control cycle.

7.2 ComputedTorqueController Class

```

1 class ComputedTorqueController:
2     def __init__(self, Kp=400.0, Kd=40.0, tau_max=10.0,
3                 pulley_radius=0.04775):
4         self.Kp = Kp
5         self.Kd = Kd
6         self.tau_max = tau_max
7         self.pulley_radius = pulley_radius
8
9     @property
10    def omega_n(self): return np.sqrt(self.Kp) # natural freq [rad/s]
11
12    @property
```

```

13     def zeta(self): return self.Kd / (2*self.omega_n) # damping
        ratio
14
15     def compute(self, q, q_dot, q_des, q_dot_ref, q_ddot_ref,
16                 M, h) -> CTControllerOutput:
17         # --- PD + feedforward ---
18         a_des = (q_ddot_ref
19                 + self.Kp * (q_des - q)
20                 + self.Kd * (q_dot_ref - q_dot))
21
22         # --- Inverse dynamics ---
23         tau_raw = M @ a_des + h
24
25         # --- Cable tension decomposition (joint 2) ---
26         F_net = tau_raw[1] / self.pulley_radius
27         T_green = max(F_net, 0.0)
28         T_red   = max(-F_net, 0.0)
29
30         # --- Saturation ---
31         tau_clip = np.clip(tau_raw, -self.tau_max, self.tau_max)
32
33         return CTControllerOutput(
34             tau_clip=tau_clip, tau_raw=tau_raw,
35             T_green=T_green, T_red=T_red,
36             q_des=q_des, a_des=a_des)

```

Listing 8: Pure NumPy CT controller — `rl/computed_torque_controller.py`

► Equations implemented by the above code

The `compute()` method executes the full CT pipeline in five lines:

1. **PD + Feedforward:** $\mathbf{a}_{\text{des}} = \ddot{\mathbf{q}}_{\text{ref}} + K_p(\mathbf{q}_{\text{des}} - \mathbf{q}) + K_d(\dot{\mathbf{q}}_{\text{ref}} - \dot{\mathbf{q}})$
2. **Inverse Dynamics:** $\boldsymbol{\tau}_{\text{raw}} = \mathbf{M}(\mathbf{q}) \mathbf{a}_{\text{des}} + \mathbf{h}(\mathbf{q}, \dot{\mathbf{q}})$
3. **Cable Decomposition:** $F_{\text{net}} = \tau_2 / r_p$, then $T_g = \max(F, 0)$, $T_r = \max(-F, 0)$
4. **Saturation:** $\boldsymbol{\tau}_{\text{clip}} = \text{clip}(\boldsymbol{\tau}, -\tau_{\text{max}}, +\tau_{\text{max}})$

The cable tension decomposition ensures the physical constraint that cables can only *pull* (unidirectional force).

7.3 EE-to-Joint Reference Conversion

The helper function `ik_to_joint_space_references()` maps end-effector references to joint space:

```

1  def ik_to_joint_space_references(
2      ee_pos_ref, ee_vel_ref, ee_acc_ref, # EE targets [m, m/s, m/s
        ^2]
3      L1, L2, # link lengths [m]
4      q_seed, # warm-start for IK [rad]
5      solve_ik_fn, # callback for 2R IK
        solver
6  ) -> Tuple[np.ndarray, np.ndarray, np.ndarray, bool]:
7      # 1. Position IK
8      q_des, ok = solve_ik_fn(L1, L2, ee_pos_ref, q_seed)

```

```

9
10     # 2. Analytical Jacobian at q_des
11     s1, c1 = np.sin(q_des[0]), np.cos(q_des[0])
12     s12 = np.sin(q_des[0] + q_des[1])
13     c12 = np.cos(q_des[0] + q_des[1])
14     J = np.array([[ -L1*s1 - L2*s12, -L2*s12],
15                  [  L1*c1 + L2*c12,  L2*c12]])
16
17     # 3. Pseudo-inverse mapping
18     J_inv = np.linalg.pinv(J)
19     q_dot_ref = J_inv @ ee_vel_ref
20     q_ddot_ref = J_inv @ ee_acc_ref    # ignores Jdot*qdot bias
21
22     return q_des, q_dot_ref, q_ddot_ref, ok

```

Listing 9: EE-to-joint conversion — `rl/computed_torque_controller.py`

► Equations implemented by the above code

The velocity/acceleration mapping uses the first-order approximation:

$$\dot{\mathbf{q}}_{\text{ref}} = \mathbf{J}^{-1} \dot{\mathbf{x}}_{\text{ref}}, \quad \ddot{\mathbf{q}}_{\text{ref}} \approx \mathbf{J}^{-1} \ddot{\mathbf{x}}_{\text{ref}}$$

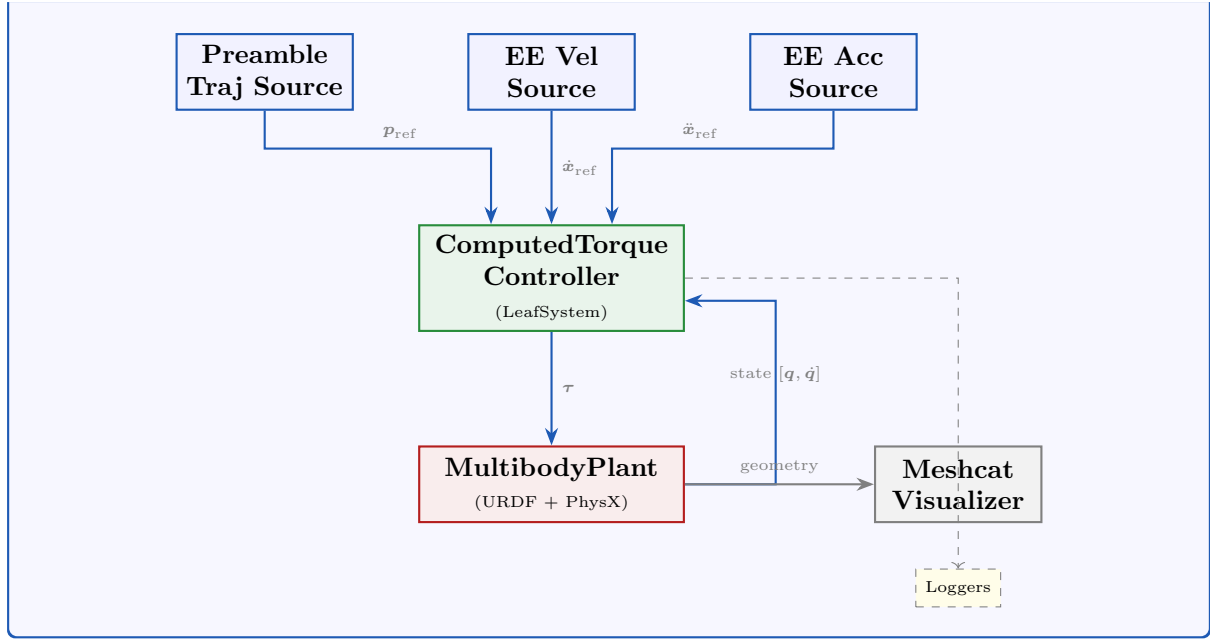
The acceleration mapping omits the $\dot{\mathbf{J}}\dot{\mathbf{q}}$ term. For slow trajectories this is negligible, but for high-speed motions the missing Coriolis coupling introduces a small feedforward error. The PD gains absorb this error in practice (< 1 mm tracking at 0.08 m/s).

8 PyDrake Architecture — Two-Plant Diagram Pattern

8.1 Drake Diagram Overview

The PyDrake implementation uses Drake's **Systems Framework** with a `DiagramBuilder` to wire signal connections between `LeafSystem` blocks:

Signal-Flow Diagram



8.2 ComputedTorqueController — Drake LeafSystem

In the PyDrake version, the controller is a `LeafSystem` with four input ports and four output ports:

Table 3: Controller I/O ports.

Direction	Port Name	Size / Description
Input	<code>desired_ee_pos</code>	2 — $[x_{\text{ref}}, y_{\text{ref}}]$
Input	<code>ee_vel_ref</code>	2 — $[\dot{x}_{\text{ref}}, \dot{y}_{\text{ref}}]$
Input	<code>ee_acc_ref</code>	2 — $[\ddot{x}_{\text{ref}}, \ddot{y}_{\text{ref}}]$
Input	<code>plant_state</code>	$n_q + n_v$ — full state vector
Output	<code>actuation</code>	2 — clipped $[\tau_1, \tau_2]$
Output	<code>joint_positions</code>	2 — desired $[q_{1,\text{des}}, q_{2,\text{des}}]$
Output	<code>torques_raw</code>	2 — pre-clip $[\tau_{1,\text{raw}}, \tau_{2,\text{raw}}]$
Output	<code>cable_tensions</code>	2 — $[T_{\text{green}}, T_{\text{red}}]$

The key computation happens in the output callback, which:

1. Reads the current state $(\mathbf{q}, \dot{\mathbf{q}})$ from the plant state port
2. Solves IK: $\mathbf{p}_{\text{ref}} \rightarrow \mathbf{q}_{\text{des}}$
3. Computes $\dot{\mathbf{q}}_{\text{ref}} = \mathbf{J}^{-1} \dot{\mathbf{x}}_{\text{ref}}$ and $\ddot{\mathbf{q}}_{\text{ref}} \approx \mathbf{J}^{-1} \ddot{\mathbf{x}}_{\text{ref}}$
4. Evaluates $\boldsymbol{\tau} = \mathbf{M} \mathbf{a}_{\text{des}} + \mathbf{h}$ using Drake's `CalcMassMatrix` and `CalcBiasTerm`
5. Clips and returns the torque

```

1 class ComputedTorqueController(LeafSystem):
2     def __init__(self, plant, manipulator, Kp, Kd, tau_max):
3         super().__init__()
4         self._plant = plant          # physics MultibodyPlant
5         self._manip = manipulator    # CupManipulatorTendon config
6         self._Kp = Kp; self._Kd = Kd; self._tau_max = tau_max

```

```

7
8      # Create internal plant context for dynamics queries
9      self._ctrl_context = plant.CreateDefaultContext()
10
11     # Declare I/O ports
12     self.DeclareVectorInputPort("desired_ee_pos", 2)
13     self.DeclareVectorInputPort("ee_vel_ref", 2)
14     self.DeclareVectorInputPort("ee_acc_ref", 2)
15     self.DeclareVectorInputPort("plant_state", plant.
16                                   num_positions()
17                                   + plant.
18                                   num_velocities
19                                   ())
20     self.DeclareVectorOutputPort("actuation", 2, self.
21                                   _calc_torque)
22     # ... additional output ports ...
23
24     def _calc_torque(self, context, output):
25         # 1. Read state
26         state = self.get_input_port(3).Eval(context)
27         self._plant.SetPositionsAndVelocities(self._ctrl_context,
28                                                state)
29         q = state[:nq]; q_dot = state[nq:]
30
31         # 2. IK: EE ref -> joint ref
32         ee_ref = self.get_input_port(0).Eval(context)
33         q_des, ok = self._manip.ik.analytical(self._plant, ee_ref,
34                                               q_seed)
35
36         # 3. Jacobian -> joint velocity/acceleration refs
37         J = self._manip.ik.jacobian_analytical(self._plant, self.
38                                                _ctrl_context)
39         J_inv = np.linalg.pinv(J)
40         q_dot_ref = J_inv @ ee_vel_ref
41         q_ddot_ref = J_inv @ ee_acc_ref
42
43         # 4. Dynamics
44         M = self._plant.CalcMassMatrix(self._ctrl_context)
45         h = self._plant.CalcBiasTerm(self._ctrl_context)
46
47         # 5. CT law
48         a_des = q_ddot_ref + Kp*(q_des - q) + Kd*(q_dot_ref - q_dot)
49         tau = M @ a_des + h
50         output.SetFromVector(np.clip(tau, -tau_max, tau_max))

```

Listing 10: Drake LeafSystem-based CT controller (simplified) — robots/cup_manipulator_tendon.py

► Equations implemented by the above code

The Drake LeafSystem executes the full CT pipeline every simulation step:

1. **IK:** $p_{\text{ref}} \xrightarrow{\text{2R analytical}} q_{\text{des}}$
2. **Differential IK:** $\dot{x}_{\text{ref}} \xrightarrow{J^{-1}} \dot{q}_{\text{ref}}, \quad \ddot{x}_{\text{ref}} \xrightarrow{J^{-1}} \ddot{q}_{\text{ref}}$
3. **PD + FF:** $a_{\text{des}} = \ddot{q}_{\text{ref}} + K_p(q_{\text{des}} - q) + K_d(\dot{q}_{\text{ref}} - \dot{q})$

4. Inverse Dynamics: $\tau = M(q) a_{\text{des}} + h(q, \dot{q})$

Drake's `CalcMassMatrix` and `CalcBiasTerm` use the full articulated-body algorithm (Featherstone), which is exact for the URDF inertia model.

9 Trajectory Generation

9.1 C^2 Cubic Spline Trajectories

All trajectories use **periodic cubic splines** (via `scipy.CubicSpline` with `bc_type='periodic'`) to ensure continuous position, velocity, and acceleration at the loop boundary:

$$p(t), \dot{p}(t), \ddot{p}(t) \in C^2 \quad \text{for all } t \quad (21)$$

9.2 Rectangle Trajectory with Velocity Profiling

The rectangle trajectory uses **non-uniform time stamps** to create slow-fast-slow velocity profiles:

1. Distribute N waypoints uniformly in **arc length** along the rectangle perimeter $P = 2(W + H)$
2. At each waypoint, compute the **desired speed** using a smoothstep blend between corner speed v_{corner} and straight speed v_{max} :

$$v(s) = v_{\text{corner}} + (v_{\text{max}} - v_{\text{corner}}) \cdot h\left(\frac{d(s)}{d_{\text{blend}}}\right) \quad (22)$$

where $d(s)$ is the distance to the nearest corner and $h(t) = t^2(3 - 2t)$ is the Hermite smoothstep.

3. Convert arc-length spacing to **time stamps** via $\Delta t_i = \Delta s / \bar{v}_i$ and rescale to the desired lap duration.

```

1 def speed(s):
2     t = np.clip(dist_corner(s) / d_blend, 0.0, 1.0)
3     return v_corner + (v_max - v_corner) * t * t * (3.0 - 2.0 * t)
4
5 # Arc-length uniform, time non-uniform
6 ds = P / N
7 for i in range(N):
8     t_raw[i+1] = t_raw[i] + ds / (0.5 * (speeds[i] + speeds[i+1]))
9 t_wp = t_raw * (lap_duration / t_raw[-1])

```

Listing 11: Velocity profiling for rectangle corners — `rl/trajectory.py`

► Equations implemented by the above code

The key idea: **uniform arc-length** $\rightarrow$ **non-uniform time**. The time between waypoints i and $i + 1$ is:

$$\Delta t_i = \frac{\Delta s}{\bar{v}_i}, \quad \bar{v}_i = \frac{v(s_i) + v(s_{i+1})}{2}$$

where $\Delta s = P/N$ is the constant arc-length spacing. The raw times are then rescaled to

match the desired lap duration T_{lap} :

$$t_i^{\text{scaled}} = t_i^{\text{raw}} \cdot \frac{T_{\text{lap}}}{t_N^{\text{raw}}}$$

9.3 Circle and Line Trajectories

In addition to the rectangle, two simpler trajectory shapes are available:

```

1 class CircleTrajectory:
2     def __init__(self, cx, cy, radius, N=60, lap_duration=10.0):
3         theta = np.linspace(0, 2*np.pi, N+1)      # uniform angular
              spacing
4         x = cx + radius * np.cos(theta)
5         y = cy + radius * np.sin(theta)
6         t = np.linspace(0, lap_duration, N+1)      # uniform time
7         self._spline = LoopingCubicTrajectory(t, np.c_[x, y],
              lap_duration)
8
9 class LineTrajectory:
10     def __init__(self, cx, cy, radius, N=60, lap_duration=10.0):
11         # Forward + reverse + closure (back-and-forth on a line
              segment)
12         x_fwd = np.linspace(cx - radius, cx + radius, N//2)
13         x_rev = np.flip(x_fwd)
14         x = np.concatenate([x_fwd, x_rev, [x_fwd[0]]]) # close the
              loop
15         y = np.full_like(x, cy)
16         # ...

```

Listing 12: Circle and line trajectory constructors — `rl/trajectory.py`

All trajectory classes share the same interface: `eval_position(t)`, `eval_velocity(t)`, `eval_acceleration(t)`. This allows the simulation to be trajectory-agnostic.

9.4 Move-to-Start Preamble

Before trajectory tracking begins, the arm moves from its initial pose to the first waypoint using a **cubic Hermite spline**:

$$\mathbf{p}(t) = \mathbf{H}(t/T_{\text{move}}) \quad \text{with} \quad \mathbf{p}(0) = \mathbf{p}_{\text{start}}, \dot{\mathbf{p}}(0) = \mathbf{0}, \mathbf{p}(T) = \mathbf{p}_{\text{end}}, \dot{\mathbf{p}}(T) = \dot{\mathbf{p}}_{\text{traj}}(0) \quad (23)$$

This ensures:

- **Zero initial velocity** (robot starts from rest)
- C^1 **continuity** at the handoff to the main trajectory (velocity-matched)

```

1 class PreambleTrajectorySource:
2     def eval_position(self, t):
3         if self._move is not None and t < self.move_duration:
4             return self._move.eval_position(t)
5         t_main = max(0.0, t - self.move_duration)
6         return self._main.eval_position(t_main)

```

Listing 13: Preamble source switches between move and main — `rl/trajectory.py`

10 Cable Routing and Tension Decomposition

10.1 Belt-Pulley Kinematics

Joint 2 is actuated through a cable (HTD 5M timing belt) routed as:

Motor $\xrightarrow{\text{belt}}$ **Drive Pulley** $\rightarrow$ **Idler (L/R)** $\rightarrow$ **Big Pulley** $\rightarrow$ **Cable Endpoint**

The relationship between cable force and joint torque:

$$\tau_2 = F_{\text{net}} \cdot r_p = (T_{\text{green}} - T_{\text{red}}) \cdot r_p \quad (24)$$

where $T_{\text{green}} \geq 0$ is the tension on the retracting side (pulling) and $T_{\text{red}} \geq 0$ is the tension on the extending side (releasing). Since cables can only pull:

$$T_{\text{green}} = \max\left(\frac{\tau_2}{r_p}, 0\right), \quad T_{\text{red}} = \max\left(-\frac{\tau_2}{r_p}, 0\right) \quad (25)$$

10.2 Cable Visualization via DrakeCablePlant

The cable geometry is computed by a headless Drake plant (`cable.py`):

```

1 from cable import DrakeCablePlant
2
3 drake_cable = DrakeCablePlant(urdf_path, q1, q2)
4 drake_cable.update(q1_new, q2_new)           # sync joint angles
5 points = drake_cable.get_cable_world_points() # [(route, pts), ...]
6 arcs    = drake_cable.get_wrap_arcs()         # pulley wrap arcs

```

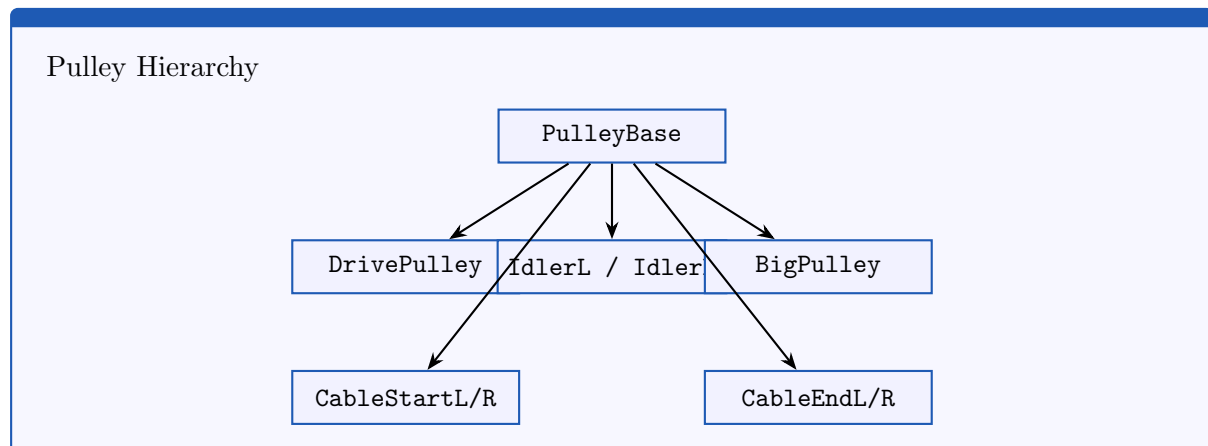
Listing 14: Cable FK pipeline.

The cable routing uses tangent-line geometry between pulleys:

- **External tangent:** when cables wrap on the same side of two pulleys
- **Internal tangent:** when cables cross between pulleys
- Tangent points computed from 2D circle-to-circle geometry, then transformed to 3D world frame via FK

10.3 Pulley Class Hierarchy

All cable-routing primitives inherit from `PulleyBase` (defined in `cable.py`). The hierarchy loads OBJ meshes via `trimesh` to auto-compute centroids and radii:



Each subclass specifies its OBJ mesh filename, URDF body name, and shaft direction. The base class lazily computes:

- **Centroid:** mesh geometric centre, rotated by URDF visual-origin RPY into body frame
- **Radius:** maximum perpendicular distance from centroid to shaft axis

```

1 class PulleyBase:
2     def _compute_centroid(self) -> np.ndarray:
3         mesh = trimesh.load(os.path.join(self.assets_dir, self.
4             obj_name))
5         R = self._R_vis()    # rotation from URDF visual-origin RPY
6         return R @ mesh.centroid + np.array(self.vis_xyz)
7
8     def _compute_radius(self) -> float:
9         mesh = trimesh.load(os.path.join(self.assets_dir, self.
10             obj_name))
11         R = self._R_vis()
12         verts_body = (R @ mesh.vertices.T).T + np.array(self.vis_xyz)
13         axis = self.shaft_axis_body
14         # Max perpendicular distance from centroid to shaft axis
15         return max(|v - centroid - proj(v-centroid, axis)| for v in
16             verts_body)

```

Listing 15: PulleyBase mesh property computation — cable.py

10.4 Tangent-Line Geometry

The cable contact points between two pulleys are computed via **circle–circle tangent lines**. Given two coplanar circles (c_1, r_1) and (c_2, r_2) separated by distance $D = \|c_2 - c_1\|$:

External tangent (cables wrap same side):

$$\begin{aligned}
 \cos \alpha &= \frac{r_1 - r_2}{D}, & \sin \alpha &= \sqrt{1 - \cos^2 \alpha} \\
 \hat{n} &= \cos \alpha \hat{d} + s \sin \alpha \hat{d}^\perp \quad (s = \pm 1) \\
 T_1 &= c_1 + r_1 \hat{n}, & T_2 &= c_2 + r_2 \hat{n}
 \end{aligned} \tag{26}$$

Internal tangent (cables cross between pulleys):

$$\begin{aligned}
 \cos \alpha &= \frac{r_1 + r_2}{D} \\
 T_1 &= c_1 + r_1 \hat{n}, & T_2 &= c_2 - r_2 \hat{n}
 \end{aligned} \tag{27}$$

The parameter $s \in \{+1, -1\}$ selects which of the two parallel tangent lines to use (upper vs. lower). These 2D contact points are computed in the XY plane, with Z preserved from each pulley’s centroid.

10.5 CableRig — Route Assembly

The **CableRig** class aggregates all pulleys and cable anchors, defining the two tendon routes:

```

1 @dataclass
2 class CableRoute:
3     segments: list          # [(waypoint_cfg, offset), ...]

```

```

4     meshcat_path: str          # e.g., "/cable/green"
5     meshcat_color: Rgba        # color for visualization
6     label: str                 # "Green (Drive->IdlerR->Big->EndL)"
7
8     class CableRig:
9         def __init__(self):
10             # Green: Start_R -> Drive -> IdlerR -> BigPulley -> EndL
11             self.cable_green = CableRoute(...)
12             # Red: Start_L -> Drive -> IdlerL -> BigPulley -> EndR
13             self.cable_red = CableRoute(...)
14
15         def compute_tangents(self, plant, plant_context, manipulator):
16             """Update all inter-pulley tangent contact points via FK."""
17             # 6 tangent pairs: Drive<->IdlerR, IdlerR<->Big, Big<->EndL
18             #                        Drive<->IdlerL, IdlerL<->Big, Big<->EndR
19             # Each uses tangent_in_world_frame() with appropriate branch/
20             # kind

```

Listing 16: Cable route definitions — cable.py

► Equations implemented by the above code

The `compute_tangents()` method must be called whenever the joint configuration changes, because cross-frame tangent pairs (e.g., IdlerR on link1 $\leftrightarrow$ BigPulley on link2) are kinematically coupled via q_2 . Each call performs 6 pairs of FK + tangent computations.

10.6 CableSpring — Optional Compliance

Optional helical springs at cable endpoints provide compliance and visual feedback:

```

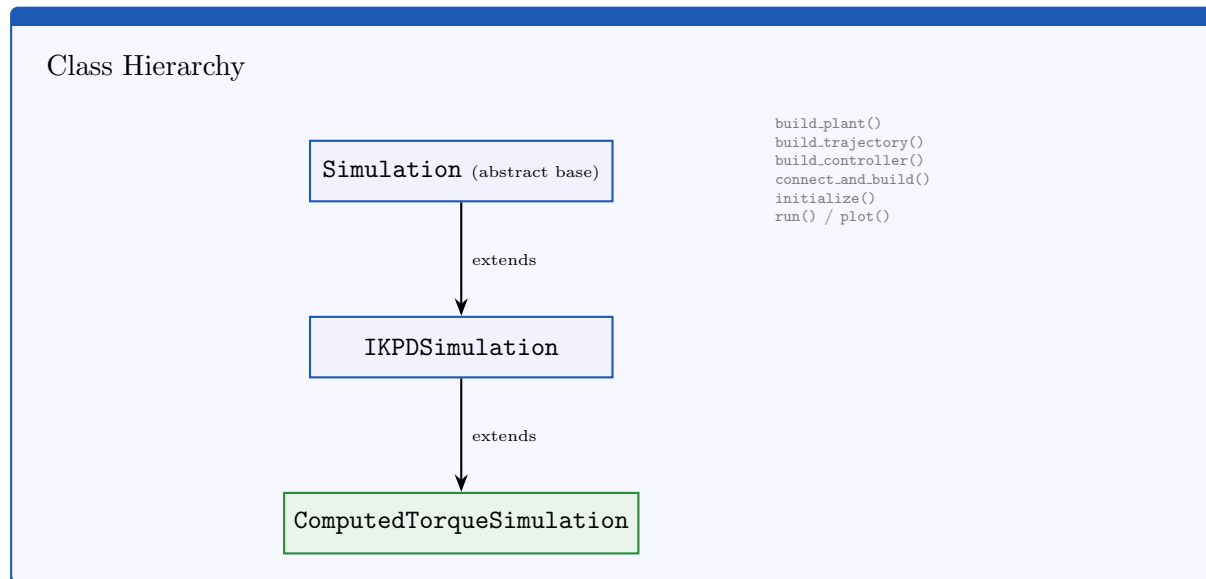
1 @dataclass
2 class CableSpring:
3     stiffness: float = 100.0      # [N/m]
4     rest_length: float = 0.002   # [m]
5     n_coils: int = 6              # helical turns
6     amplitude: float = 0.004     # helix radius [m]
7     enabled: bool = True
8
9     def spring_zigzag_points(p_start, p_end, n_coils, amplitude):
10         """Generate 3D helix for Meshcat visualization."""
11         ax = (p_end - p_start) / np.linalg.norm(p_end - p_start)
12         perp1, perp2 = orthonormal_frame(ax)
13         # p(t) = p_start + ax*t + amplitude*(cos*perp1 + sin*perp2)

```

Listing 17: Spring visualization — cable.py

11 Simulation — Builder Pattern

11.1 ComputedTorqueSimulation Class Hierarchy



11.2 Builder Pattern — Step-by-Step

The simulation follows a strict sequential build:

1. `build_plant()` — Load URDF into `MultibodyPlant`, add actuators, finalize:

```

1 self.builder = DiagramBuilder()
2 self.plant, self.scene_graph = \
3     AddMultibodyPlantSceneGraph(self.builder, time_step=self._DT)
4 parser = Parser(self.plant)
5 parser.AddModels(urdf_path)
6 self.manipulator.add_joint_actuators(self.plant)
7 self.plant.Finalize()
  
```

2. `build_trajectory()` — Create C^2 cubic spline via `PiecewisePolynomial`:

```

1 self.traj_ref = PiecewisePolynomial.
2     CubicWithContinuousSecondDerivatives(
3         t_wp, xy_wp.T, periodic=True)
4 self.traj_vel_ref = self.traj_ref.derivative(1)
5 self.traj_acc_ref = self.traj_ref.derivative(2)
  
```

3. `build_controller()` — Add `CTController` as `LeafSystem`
4. `connect_and_build()` — Wire all ports + loggers, call `builder.Build()`
5. `initialize()` — Set initial state, patch zero-mass bodies
6. `run()` — Call `simulator.AdvanceTo()` in chunks with lap reporting

11.3 IKPDSimulation — Base Closed-Loop Class

`IKPDSimulation` provides the PD control baseline and the full build pipeline. Its constructor accepts joint-space PD gains:


```

1 class IKPDSimulation:
2     _DT = 0.002    # 500 Hz simulation timestep
3
4     def __init__(self, manip_config, meshcat=None,
5                   Kp1=1000.0, Kd1=100.0,          # J1 gains [Nm/rad]
6                   Kp_cable=100.0, Kd_cable=10.0): # J2 cable gains [N/
7                                     m]
8         self.manipulator = CupManipulatorTendon(manip_config)
9         self._meshcat = meshcat
10        # ... store gains ...

```

Listing 18: IKPDSimulation constructor.

11.4 ComputedTorqueSimulation — CT Subclass

The CT subclass overrides `build_controller()` and `connect_and_build()` to add EE velocity and acceleration reference sources:

```

1 class ComputedTorqueSimulation(IKPDSimulation):
2     def __init__(self, manip_config, meshcat=None,
3                   Kp=400.0, Kd=40.0, tau_max=10.0):
4         super().__init__(manip_config, meshcat)
5         self._ct = ComputedTorqueController(Kp, Kd, tau_max,
6                                             PULLEY_RADIUS)
7
8     def build_controller(self):
9         # Adds ComputedTorqueController as Drake LeafSystem
10        # with 4 input ports and 4 output ports
11
12    def connect_and_build(self):
13        # Wires: [pos_ref, vel_ref, acc_ref, plant_state] -> CT ->
14               plant
15        # Plus 8 loggers for diagnostic plotting

```

Listing 19: CT simulation overrides.

11.5 Initialization Priority

The `initialize()` method determines starting joint angles using a priority chain:

1. `--home-ee [x y]` → IK solve for joint angles (CLI override)
2. `--home-joints [q1 q2]` → direct angles in degrees (CLI override)
3. Preamble start pose from `build_move_to_start()`
4. IK of first trajectory waypoint
5. Hard fallback: $[5^\circ, 15^\circ]$

11.6 Cable Visualization Loop

During `run()`, cable geometry is updated every simulation chunk (0.1 s):

```

1 def run(self, traj_shape=''):
2     while True:
3         self.simulator.AdvanceTo(t + 0.1)    # 100 ms chunks

```

```

4      # Update cable tangent contacts at new configuration
5      self.manipulator.compute_tangents(self.plant, self.plant_ctx)
6      # Redraw cable polylines in Meshcat
7      draw_cables(self._meshcat, self.plant, self.plant_ctx,
8                  self.manipulator, self.manipulator.rig)

```

Listing 20: Cable visualization update in the simulation loop.

11.7 Zero-Mass Body Patching

Some URDF parts exported from Onshape have **zero mass** (no material assigned). Drake's SAP solver produces NaN if a body has zero inertia. The simulation patches these bodies with a nominal inertia:

```

1  _M_PATCH = SpatialInertia(
2      mass=0.3,
3      p_PScm_E=np.zeros(3),
4      G_SP_E=UnitInertia(1e-2, 1e-2, 1e-2),
5  )
6
7  for body_idx in range(plant.num_bodies()):
8      body = plant.get_body(body_idx)
9      if body.default_mass() < 1e-10 and body.name() != "world":
10         body.SetSpatialInertiaInBodyFrame(context, _M_PATCH)

```

Listing 21: Zero-mass body fix — `Simulation._patch_zero_mass_bodies()`

12 Simulation Modes and CLI

The script is controlled via command-line arguments dispatched through a registry:

Table 4: Simulation modes.

--mode	Class	Description
pd-control	IKPDSimulation	IK → joint-space PD (no feedforward)
computed-torque	ComputedTorqueSimulation	Full CT with feedforward
scene-viz	—	Static scene in Meshcat
scene-viz-q	—	Interactive joint sliders
ee-trajectory	—	IK visualization with EE path

```

1  _SIM_REGISTRY = {
2      'pd-control': (IKPDSimulation, lambda a: {}, "PD CONTROL"),
3      'computed-torque': (
4          ComputedTorqueSimulation,
5          lambda a: dict(Kp=a.ct_kp, Kd=a.ct_kd, tau_max=a.ct_tau_max),
6          "COMPUTED TORQUE"
7      ),
8  }
9
10 # In main():

```

```

11 Cls, kwargs_fn, label = _SIM_REGISTRY[args.mode]
12 sim = Cls(config, meshcat=meshcat, **kwargs_fn(args))
13 sim.build_plant()
14 sim.build_trajectory(args)
15 sim.build_move_to_start(args)
16 sim.build_controller()
17 sim.connect_and_build()
18 sim.initialize(home_override)
19 sim.run(traj_shape=args.traj_shape)
20 sim.plot(traj_shape=args.traj_shape)

```

Listing 22: Mode registry dispatch.

13 Visualization — Meshcat

PyDrake uses **Meshcat** for real-time 3D visualization in the browser:

- Robot geometry rendered from URDF meshes (OBJ format)
- Joint frames visualized with coordinate triads
- Cable routing drawn as colored polylines (green/red for each tendon)
- Camera configurable via `--meshcat-azimuth`, `--meshcat-elevation`, `--meshcat-distance`

```

1 meshcat = StartMeshcat()
2 print(f"Meshcat: {meshcat.web_url()}") # http://localhost:7000
3
4 MeshcatVisualizer.AddToBuilder(
5     builder, scene_graph, meshcat,
6     MeshcatVisualizerParams(role=Role.kIllustration))

```

Listing 23: Meshcat setup.

13.1 Coordinate Frame Visualization

The `project_utils/viz.py` module provides frame triad visualization for debugging kinematic chains:

```

1 def add_frames_to_meshcat(meshcat, plant, context, manipulator=None):
2     """Add RGB coordinate-frame triads for all bodies."""
3     for i in range(plant.num_frames()):
4         frame = plant.get_frame(FrameIndex(i))
5         if frame.name() == "world":
6             continue
7         X_WF = plant.CalcRelativeTransform(context,
8             plant.world_frame(), frame)
9         add_frame_triad(meshcat, f"/frames/{frame.name()}",
10             length=0.05, X_WF=X_WF)
11
12 def add_frame_triad(meshcat, path, length, X_WF):
13     """Draw XYZ cylinder triad at pose X_WF."""
14     # X-axis: red cylinder, rotated pi/2 around Y
15     # Y-axis: green cylinder, rotated -pi/2 around X
16     # Z-axis: blue cylinder (no rotation)

```

Listing 24: Frame triad helper — `project_utils/viz.py`

13.2 Camera Control

Spherical camera positioning via CLI arguments:

```

1 def set_meshcat_camera_spherical(meshcat, azimuth_deg, elevation_deg,
2                                   distance=3.0, target=np.zeros(3)):
3     az, el = np.deg2rad(azimuth_deg), np.deg2rad(elevation_deg)
4     x = distance * np.cos(el) * np.cos(az)
5     y = distance * np.cos(el) * np.sin(az)
6     z = distance * np.sin(el)
7     meshcat.SetCameraPose(target + [x, y, z], target)

```

Listing 25: Spherical camera setup — `project_utils/viz.py`

14 Results and Plotting

14.1 3×3 Diagnostic Plot

The `plot()` method generates a comprehensive 3×3 figure:

Table 5: Plot layout: rows $\times$ columns.

	Col 0: Position	Col 1: Velocity	Col 2: Acceleration
Row 0: EE	x, y actual vs ref	$\dot{x}, \dot{y}$	$\ddot{x}, \ddot{y}$
Row 1: Joints	q_1, q_2 [deg]	$\dot{q}_1, \dot{q}_2$	$\ddot{q}_1, \ddot{q}_2$
Row 2: Forces	τ required vs applied	Cable tensions	EE XY path

Key derived signals:

- EE velocity *actual* computed via $\dot{\mathbf{p}}_{EE} = \mathbf{J}(\mathbf{q}) \dot{\mathbf{q}}$ (no finite differences)
- EE acceleration actual via `np.gradient()` of velocity signal
- Joint-space references via $\mathbf{J}^{-1}$ of logged EE references

Insert computed-torque results plot here.

`plots/computed_torque_rect_*.png`

Figure 2: Example 3×3 CT results for rectangle trajectory. Replace with actual `.png` from `plots/` directory.

14.2 Key Metrics

EE tracking RMS	< 1 mm (after move-to-start preamble)
Peak torque τ_1	≈ 5 Nm (within $\tau_{\max} = 10$ Nm budget)
Peak cable tension	≈ 100 N
Simulation timestep	2 ms (500 Hz)
Real-time factor	> 10 $\times$ on CPU (no GPU needed)

15 Notation Summary

Table 6: Notation reference.

Symbol	Description	Unit
$\mathbf{q} = [q_1, q_2]^\top$	Joint positions	rad
$\dot{\mathbf{q}} = [\dot{q}_1, \dot{q}_2]^\top$	Joint velocities	rad/s
$\ddot{\mathbf{q}}$	Joint accelerations	rad/s ²
$\mathbf{M}(\mathbf{q})$	Mass (inertia) matrix	kg·m ²
$\mathbf{h}(\mathbf{q}, \dot{\mathbf{q}})$	Bias vector = $\mathbf{C}\dot{\mathbf{q}} + \mathbf{g}$	Nm
$\boldsymbol{\tau}$	Applied joint torques	Nm
$\mathbf{J}(\mathbf{q})$	Analytical Jacobian	m/rad
$\mathbf{p}_{\text{EE}}$	End-effector position	m
K_p, K_d	PD gains	s ⁻² , s ⁻¹
ω_n, ζ	Natural frequency, damping ratio	rad/s, —
r_p	Pulley pitch radius	m
$T_{\text{green}}, T_{\text{red}}$	Cable tensions	N
L_1, L_2	Link lengths	m

16 Running the Simulation

```

1  # Activate environment
2  conda activate pydrake
3
4  # Default: computed-torque with rectangle trajectory
5  python
6      script_cup_manipulator_pendulam_tendon_with_spring_damper_pydrake.
7      py \
8      --mode computed-torque --duration 10.0
9
10 # Custom gains and circle trajectory
11 python
12     script_cup_manipulator_pendulam_tendon_with_spring_damper_pydrake.
13     py \
14     --mode computed-torque \
15     --ct-kp 400 --ct-kd 40 --ct-tau-max 10 \
16     --traj-shape circle --traj-cx 0.4 --traj-radius 0.08
17
18 # PD control (no feedforward)

```

```

15 python
    script_cup_manipulator_pendulam_tendon_with_spring_damper_pydrake.
16 py \
    --mode pd-control --duration 10.0

```

17 Summary

Implementation Summary

1. **Dynamics:** Drake `MultibodyPlant` provides exact $\mathbf{M}(\mathbf{q})$ and $\mathbf{h}(\mathbf{q}, \dot{\mathbf{q}})$ from URDF via Featherstone's algorithm
2. **Controller:** Custom `LeafSystem` implementing $\boldsymbol{\tau} = \mathbf{M} \mathbf{a}_{\text{des}} + \mathbf{h}$ with PD + feedforward
3. **IK:** Closed-form 2R analytical solution with elbow-sign selection and seed warm-starting
4. **Trajectory:** C^2 periodic cubic splines with velocity profiling at corners
5. **Cable Model:** Physical tension decomposition $\tau_2 = (T_g - T_r) \cdot r_p$ with visualization
6. **Architecture:** Drake diagram builder pattern with signal flow and data loggers

Source files:

- `script_cup_manipulator_pendulam_tendon_with_spring_damper_pydrake.py` — Main script
- `robots/cup_manipulator_tendon.py` — Robot config, Drake `LeafSystems` (IK, CT)
- `rl/computed_torque_controller.py` — Pure-NumPy CT controller (engine-agnostic)
- `rl/trajectory.py` — Trajectory classes (Rect, Circle, Line, Preamble)
- `cable.py` — `DrakeCablePlant` for cable geometry
- `project_utils/viz.py` — Meshcat visualization helpers